What Transfers to a New Benchmark? Priors, Not Item Estimates, for Label-Efficient LLM Evaluation

Arshia Teymourtash[*]
KTH Royal Institute of Technology
Stockholm, Sweden
arturt@kth.se

Ankit Grover[*]
KTH Royal Institute of Technology
Stockholm, Sweden
agrover@kth.se

Abstract

Evaluating an LLM on every item of every benchmark is expensive, so predicting item outcomes from a few revealed answers is attractive, but only if what is learned from past benchmarks carries over to new ones. We ask what does, in a controlled comparison where the models, the evaluation tasks and the order in which answers are revealed are held fixed and only the source of the prior changes. A prior learned from past benchmarks *transfers* to a new benchmark. Estimates of individual item difficulty *do not*. The prior's benefit is small on average and depends on the benchmark: it helps on benchmarks of *typical* difficulty and hurts on much easier or much harder ones. A neural network that predicts item difficulty from item text adds little beyond the prior. Whether pooling answers across LLMs helps *remains open*, because our evaluation reveals answers from one LLM per benchmark. Our entry to the Predictive AI Evaluation Competition (Codabench submission 1005155) is a Beta-Binomial predictor. We release the simulation code, the fixed evaluation samples and every saved score on GitHub.

1 Introduction

Evaluating language models on every item of every benchmark is expensive, so a predictor that infers a model's per-item results from a handful of labelled answers is attractive. The natural recipe is to learn from past benchmarks how items behave and carry that to a new benchmark. Whether it carries is an empirical question, and the answer matters because new benchmarks and new models keep arriving. Most benchmark-prediction methods lose their advantage over random sampling when extrapolating to higher-performing models [36], and amortized item-response models learn item difficulty from item text [35]. We ask what, among the things one can fit offline, actually transfers to an unseen benchmark.

We use the Predictive AI Evaluation Competition's setting: a *task* is one subject (a model under given evaluation settings) answering one benchmark, labels are bought one at a time, and predictions are scored by Brier ALC at 0, 1, 3, 7, 15 and 31 labels. Our contributions:

- **A controlled comparison.** Models, tasks and label order are fixed and only the source of the prior changes, in an online simulation of the competition protocol (Section 4).
- **What transfers.** A learned prior transfers to a new benchmark and item-level estimates do not. The prior helps on benchmarks of typical difficulty and hurts on much easier or harder ones, and item text adds almost nothing beyond it (Section 5).

The simulation code, the fixed evaluation samples and every saved score are on GitHub.

[*]Work done equally.

40th Conference on Neural Information Processing Systems (NeurIPS 2026), The Predictive AI Evaluation Competition at NeurIPS 2026.

2 Related work

Label-efficient evaluation. Label-efficient methods estimate LLM performance from a small number of responses. tinyBenchmarks uses curated item subsets and item-response models to reproduce aggregate benchmark results [24]. Work on predicting results for new models finds that most benchmark-prediction methods lose their advantage over random sampling when extrapolating to higher-performing models, while one estimator that combines predictions with random labels keeps it [36]. That work estimates aggregate benchmark performance, not per-item outcomes on new benchmarks, so we treat it as related evidence for our question and not as an answer to it. We predict on entirely held-out benchmarks, and our target is the probability that each unanswered item is answered correctly, given a few revealed labels. Labels arrive in a random order, so we do not study which items to label next. The question is what carries over to a held-out benchmark.

Item response theory. It separates subject ability from item difficulty, with Rasch as its basic model [29]. Latent-space and multilingual extensions address interactions and language effects [17, 23]. Most directly related, a model that predicts item difficulty from item text, trained through the item-response likelihood on pooled responses, amortizes the cost of calibrating a benchmark [35]. Text-predicted difficulty can assign parameters to previously unobserved questions, but its usefulness on an entirely new benchmark requires empirical validation. We compare simple probes of this kind (ridge regression and a small neural network on item embeddings, not the model above) with priors that carry over only the typical accuracy of a task, which separates information shared across benchmarks from item-level estimates. Item response theory gains most when many LLMs answer the same items, and our draws reveal answers from one LLM per benchmark, so sharing labels across LLMs is not tested.

Empirical Bayes and shrinkage. Both carry population-level information to related tasks by learning a prior across them [30, 9, 28]. In our comparison, only the prior fitted on training benchmarks is empirical Bayes. The fixed Jeffreys prior [16] and the settings tuned on validation are separate prior sources.

Robust priors and prediction-powered inference. Robust meta-analytic-predictive priors give a statistical precedent for mixture-based protection against prior–data conflict [32], although their clinical-trial setting and construction differ from ours. Prediction-powered inference combines predictions with observed labels, but targets valid inference for population quantities [5].

3 Data and evaluation protocol

Data. The data are a binary response matrix $Y \in \{0,1\}^{N \times M}$ with N subjects (rows, index i) and M items (columns, index j): $Y_{ij} = 1$ if subject i answers item j correctly. Most entries are missing, since a subject answers only the items of the benchmarks it was evaluated on. Each item j belongs to the dataset $d(j)$ of one of $D = 17$ benchmarks, so a benchmark is a block of columns, and a *task* is one row restricted to one block: subject i on benchmark d.

Each item j has an embedding $x_j \in \mathbb{R}^{1024}$ computed once from its text by the frozen BGE-M3 encoder [7] with a context of 8,192 tokens, and normalised to unit length. We use BGE-M3 because the corpus is multilingual (it includes Chinese, Russian and several African languages) and some items are long, which English-only and 512-token encoders handle poorly.

Online simulation and splits. The competition protocol is *online*: for each task, labels are revealed one at a time and the predictor updates from them. Whatever the predictor knows before the first label is *offline*, that is, fitted on the training benchmarks beforehand. Our *online simulation* runs this protocol on recorded responses, with disjoint column blocks for training, validation, and test (11, 3, and 3 benchmarks): priors are fitted offline on training, and validation selects settings. The validation and test benchmarks are *out of distribution* (OOD): their responses are excluded from offline fitting.

A *competition-shaped draw* samples one subject per benchmark, so a predictor cannot borrow labels from other subjects on the same benchmark; this simulates the competition protocol on public data. As a separate *in-distribution* (ID) evaluation, we hold out rows (subjects) within the training blocks,

so its benchmarks and items have been seen during fitting, and retain the questions covered by the fit subjects. The offline comparator reuses its fitted benchmark rates and item difficulties where available, and its online update changes only the subject ability.

Budgets and scoring. For each subject–benchmark pair we freeze a split of distinct items into acquisition and evaluation pools, assigning the extra item to acquisition when the count is odd. A frozen random acquisition order supplies nested prefixes of 0, 1, 3, 7, 15, and 31 labels. At each budget the predictor conditions on the allowed prefix and predicts the same evaluation items.

Evaluation outcomes are used only to score those predictions. We average Brier scores over query items, then average repeated appearances within each task and give tasks equal weight. Brier ALC combines the six budgets with weights $(0.1, 0.2, 0.2, 0.2, 0.2, 0.1)$. Paired intervals resample whole tasks with their matched predictions.

4 Methodology

We first define the three predictors, then two ways of improving on them, adaptive priors and item-text probes, and finally the sources of the prior parameters that every predictor needs.

4.1 Models

The three predictors differ in what they model: a task's accuracy (Beta-Binomial), a subject's ability and each item's difficulty (Rasch), and the similarity between items (kernel Rasch).

Beta-Binomial. Let p be a task's accuracy, with a Beta prior $p \sim \mathrm{Beta}(\alpha, \beta)$. After n revealed labels, k of them correct, the posterior is $\mathrm{Beta}(\alpha + k, \beta + n - k)$ and the prediction, the same for every item of the task, is its mean

$$\hat{p} = \frac{\alpha + k}{\alpha + \beta + n}. \tag{1}$$

The prior acts like $\alpha + \beta$ extra labels, α of them correct. The Jeffreys prior is $\alpha = \beta = \frac{1}{2}$ [16]. This is the exact Bayesian sequential update; nothing is learned.

Rasch. The probability that subject i answers item j correctly is

$$p_{ij} = \sigma(\theta_i - b_j), \qquad b_j = \delta_j - \mu_{d(j)}, \tag{2}$$

where σ is the logistic function, θ_i is the subject's ability, and b_j is the item's difficulty. This is the Rasch model [29] with the item difficulty split into a benchmark part and an item part: δ_j is the item's deviation from the average difficulty $-\mu_d$ of its benchmark $d = d(j)$, so a larger benchmark rate μ_d means an easier benchmark. The priors are $\mu_d \sim \mathcal{N}(m_0, \sigma_\mu^2)$, $\theta_i \sim \mathcal{N}(0, \sigma_\theta^2)$ and $\delta_j \sim \mathcal{N}(0, \sigma_\delta^2)$. At every budget we refit the maximum a posteriori (MAP) estimate from scratch on all labels revealed so far in the draw; an unlabelled item gets $\delta_j = 0$.

Kernel Rasch. Kernel Rasch keeps Equation (2) and changes only the prior on the item deviations $\boldsymbol{\delta} = (\delta_1, \ldots, \delta_M)^\top$. Rasch treats them as independent Gaussians, $\delta_j \sim \mathcal{N}(0, \sigma_\delta^2)$; kernel Rasch makes them jointly Gaussian, with covariance given by the similarity of the item embeddings:

$$\boldsymbol{\delta} \sim \mathcal{N}\big(\mathbf{0},\ K\big), \qquad K = \sigma_\delta^2\big[(1-\rho)\, I + \rho\, S\big], \quad S_{jj'} = \exp\Big(\frac{\cos(x_j, x_{j'}) - 1}{T}\Big), \tag{3}$$

where x_j are the item embeddings of Section 3, $\rho \in [0, 1]$ weights the similarity part S and T is its temperature. At $\rho = 0$ the two priors coincide; for $\rho > 0$, similar items share difficulty, while items of different benchmarks stay independent. After the refit, a labelled item keeps its fitted deviation, and an unlabelled item j receives the posterior mean given the labelled items L of its benchmark, $\hat{\delta}_j = \rho\, \sigma_\delta^2\, S_{jL} K_{LL}^{-1} \hat{\boldsymbol{\delta}}_L$, which is 0 at $\rho = 0$ as in Rasch. With one subject per benchmark, as in the competition-shaped draws, no labels are shared across subjects: the joint fit equals independent per-task fits (maximum difference 10^{-14} in our check), so Rasch there differs from the Beta-Binomial only in its prior.

Offline model that keeps its fitted values. For the ID evaluation, the offline Rasch keeps its fitted μ_d and δ_j for a benchmark and items it has seen and updates only the subject ability θ_i online (a one-dimensional MAP on the task's own labels). For an unseen benchmark or item it falls back to m_0 and 0, i.e. to the fitted prior alone.

4.2 Adaptive priors

The predictors above keep their prior fixed whatever labels arrive. An *adaptive prior* lets the first labels change it. We test three closed-form versions, each refit without any held-out benchmark.

Beta mixture. The prior is a mixture of $K \in \{2, 3\}$ Beta distributions with weights w_c, fitted by expectation maximisation on the accuracies of the training tasks. After k of n labels are correct, the weight of each component is updated by how well it explains them, and the prediction is the weighted mean of the components' posterior means,

$$w'_c \propto w_c \, \frac{\mathrm{B}(\alpha_c + k,\ \beta_c + n - k)}{\mathrm{B}(\alpha_c,\ \beta_c)}, \qquad \hat{p} = \sum_c w'_c \, \frac{\alpha_c + k}{\alpha_c + \beta_c + n}, \tag{4}$$

where B is the Beta function.

Averaging over the prior. Instead of one (α, β), we average the Beta-Binomial prediction over a grid of prior parameters, weighting each by the same factor $\mathrm{B}(\alpha + k, \beta + n - k)/\mathrm{B}(\alpha, \beta)$. We average over the prior's strength $\alpha + \beta$ at the fitted mean, or over its mean and strength together, in which case nothing is learned from training.

Switch to Jeffreys. Once at least $k_0 \in \{3, 7\}$ labels are revealed, we predict with the Jeffreys prior if the accuracy so far, k/n, lies more than η from the fitted prior's mean $\alpha/(\alpha + \beta)$, and with the fitted prior otherwise. We choose η on validation.

4.3 Probes

The predictors above ignore what an item says. A probe predicts an item's relative difficulty from its embedding before any response from that benchmark is available. We compare ridge regression on the embeddings, a probe using log character count, and a random score. We add ridge regression and a small neural network (MLP) that fit either the item pass rate or the negative fitted Rasch difficulty (easiness), each centred within its benchmark. The fitted probes use a fixed sample of 8,462 items from the 11 training benchmarks, at most 800 per benchmark, and choose their regularisation by five folds grouped by training benchmark. Appendix H gives the training details.

We score a probe on each benchmark d by two numbers and average each over benchmarks. Let y_j be the observed pass rate of item j across recorded subjects and $\hat{y}_j$ its predicted easiness, so higher values mean easier items. The *Spearman correlation* $\rho_d = \mathrm{corr}\big(\mathrm{rank}(\hat{y}_j), \mathrm{rank}(y_j)\big)$ compares the two orderings of the items: it is 1 if the probe orders them perfectly, 0 if its order is unrelated to the truth, and -1 if it is reversed. The *pairwise accuracy* $A_d = \frac{1}{|P_d|} \sum_{(j,k) \in P_d} s_{jk}$ is the share of item pairs the probe orders correctly, where P_d is the set of pairs with $y_j \neq y_k$ (observed ties are excluded), $s_{jk} = 1$ if $\hat{y}_j - \hat{y}_k$ has the same sign as $y_j - y_k$, $s_{jk} = \frac{1}{2}$ if the probe predicts a tie, and $s_{jk} = 0$ otherwise. A random score has $A_d = 0.5$ on average.

To test whether a probe helps prediction, we give the item deviations of Rasch a prior mean taken from it,

$$\delta_j \sim \mathcal{N}\big(\gamma \, z_j,\ \sigma_\delta^2\big), \tag{5}$$

where z_j is the probe's predicted difficulty of item j, centred within its benchmark (the benchmark rate μ_d absorbs the average), and γ is a scale. Setting $\gamma = 0$ gives the fitted Rasch. At prediction time the probes use only item features. Training outcomes supply supervision, and validation and test outcomes are used only for evaluation and for choosing γ on validation.

4.4 Priors

Every predictor above needs prior parameters, and where they come from is the one thing our comparison changes. We set them from one of three sources. *Fixed in advance* means nothing is learned,

so the Beta-Binomial uses Jeffreys and the Rasch models use unit widths. *Tuned on validation* means the best setting on the three validation benchmarks from a grid. *Fitted on training* means estimated from the 11 training benchmarks only. Table 1 lists the settings.

Table 1: Prior parameters by model and source. The prior mean of the Beta-Binomial is $\alpha/(\alpha+\beta)$ and its strength is $\alpha+\beta$. Kernel Rasch keeps ρ and T from validation, so its kernel is not fitted on training.

Model	Fixed in advance	Tuned on validation	Fitted on training
Beta-Binomial	$\alpha=\beta=\frac{1}{2}$	$\alpha=0.4$, $\beta=0.6$	$\alpha=1.92$, $\beta=1.76$
Rasch	$m_0=0$, $\sigma_\mu=\sigma_\theta=\sigma_\delta=1$	$m_0=0.048$, $\sigma_\mu=2$, $\sigma_\theta=0.5$, $\sigma_\delta=1$	$m_0=0.175$, $\sigma_\mu=0.75$, $\sigma_\theta=1.38$, $\sigma_\delta=1.49$
Kernel Rasch	as Rasch, $\rho=0.5$, $T=0.1$	$m_0=0.048$, $\sigma_\mu=2$, $\sigma_\theta=1$, $\sigma_\delta=1.4$, $\rho=0.8$, $T=0.2$	as fitted Rasch, $\rho=0.8$, $T=0.2$

Fitted on training means estimated from the responses of the 11 training benchmarks, once, before any simulation starts. For the Beta-Binomial, we choose (α,β) to maximise the probability of the training tasks' counts, where a task with k correct labels out of n has probability proportional to $\mathrm{B}(\alpha+k,\beta+n-k)/\mathrm{B}(\alpha,\beta)$, and every benchmark counts equally. This gives a prior mean of 0.522 worth $\alpha+\beta=3.68$ labels. For Rasch, we first fit the model to 4.34 million training responses (4,809 subjects, 109,987 items) by maximum a posteriori estimation. We then set each prior parameter from the fitted values: m_0 is the mean of the benchmark rates μ_d, and each width is the spread of the corresponding fitted quantity (μ_d, θ_i or δ_j), averaged per benchmark so that the benchmark with the most subjects does not dominate. We repeat the two steps until the values stop changing. Only these prior parameters transfer to a new benchmark. The fitted μ_d, θ_i and δ_j belong to training benchmarks, subjects and items.

Tuned on validation is different, because nothing is estimated from a likelihood. We try a grid of settings, run the online simulation on the three validation benchmarks with each, and keep the setting with the lowest Brier ALC. The scale γ of the probe prior mean (Section 4.3) is chosen the same way. The two sources differ in what they learn from and what they optimise. Fitting on training uses many benchmarks and a likelihood and never sees validation or test outcomes. Tuning on validation uses three benchmarks and the score we care about and never sees test outcomes. Both are fixed before the test simulation starts, and the probes are trained on the training benchmarks' items only.

5 Results

Every difference is written Δ_{a-b}, the Brier ALC of model a minus that of model b, so negative favours a; paired 95% bootstrap intervals are over tasks. J is the Jeffreys Beta-Binomial; BB, R and KR are the Beta-Binomial, Rasch and kernel Rasch models, subscripted fix, tune or fit for the source of their prior, and R off is the offline Rasch that keeps its fitted values.

5.1 Prior source matters, and no source wins on every split

Table 2 compares the nine combinations. The validation-tuned Beta-Binomial prior is *worse* than Jeffreys on test ($\Delta_{\mathrm{BB\,tune-J}}=+0.0055$) though it is the best Beta-Binomial on validation. Every comparison between prior sources changes sign between test and validation (Appendix B); fresh draws change no result by more than 0.0007 (Appendix C).

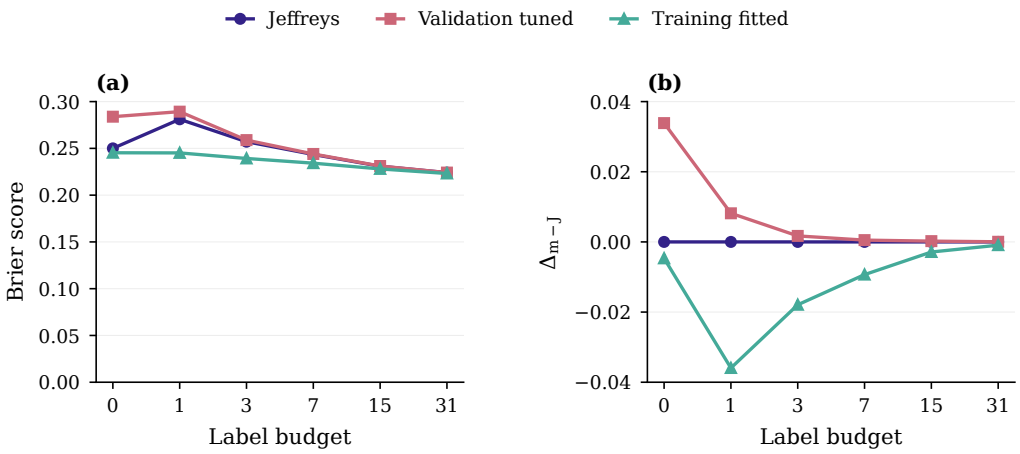


Figure 1: **The prior's source matters at zero to three labels and has faded by 31.** Brier score (a) and $\Delta_{m-\mathrm{J}}$ (b) per label budget on the three unseen test benchmarks, competition-shaped draws, for the Beta-Binomial with a fixed (Jeffreys, solid), validation-tuned (dashed) and training-fitted (dotted) prior. Jeffreys jumps to 0.75 or 0.25 after one answer, so the gap is largest at one label; at 31 labels the three differ by less than 0.001.

Table 2: **A prior fitted on training helps on the test benchmarks and hurts on the validation benchmarks; a fixed prior with strong shrinkage does nearly as well on test.** Each cell is $\Delta_{m-\mathrm{J}}$ for the model m in the row, competition-shaped draws. ID: held-out subjects on the 11 training benchmarks, 114 tasks, where the training-fitted Beta-Binomial prior is refit without those subjects. OOD validation: 3 unseen benchmarks, 80 tasks. OOD test: 3 unseen benchmarks, 45 tasks. Bold: best of the nine models in each column; underlined: second best.

Model m	ID	OOD validation	OOD test
Fixed in advance			
Beta-Binomial (Jeffreys)	0 (reference)	0 (reference)	0 (reference)
Rasch	-0.0073 $[-0.0091, -0.0055]$	$+0.0031$ $[-0.0005, +0.0066]$	-0.0102 $[-0.0118, -0.0085]$
Kernel Rasch	-0.0072 $[-0.0087, -0.0058]$	$+0.0029$ $[-0.0008, +0.0065]$	-0.0105 $[-0.0121, -0.0088]$
Tuned on validation			
Beta-Binomial	$+0.0041$ $[+0.0027, +0.0055]$	$\mathbf{-0.0015}$ $[-0.0040, +0.0010]$	$+0.0055$ $[+0.0038, +0.0072]$
Rasch	-0.0024 $[-0.0028, -0.0019]$	$+0.0004$ $[-0.0006, +0.0014]$	-0.0032 $[-0.0035, -0.0029]$
Kernel Rasch	$+0.0001$ $[-0.0004, +0.0005]$	$\underline{-0.0003}$ $[-0.0010, +0.0004]$	-0.0024 $[-0.0027, -0.0021]$
Fitted on training			
Beta-Binomial	$\mathbf{-0.0090}$ $[-0.0115, -0.0063]$	$+0.0079$ $[+0.0024, +0.0136]$	$\mathbf{-0.0138}$ $[-0.0158, -0.0116]$
Rasch	$\underline{-0.0081}$ $[-0.0101, -0.0060]$	$+0.0052$ $[+0.0008, +0.0097]$	$\underline{-0.0119}$ $[-0.0133, -0.0106]$
Kernel Rasch	-0.0063 $[-0.0076, -0.0049]$	$+0.0036$ $[-0.0007, +0.0076]$	-0.0118 $[-0.0131, -0.0105]$

5.2 Learned priors transfer to unseen benchmarks; item-level estimates do not

On the ID evaluation set the offline model that keeps its fitted values scores $\Delta_{\mathrm{R\,off}-\mathrm{J}} = -0.0852$ $[-0.0955, -0.0747]$ against $\Delta_{\mathrm{BB\,fit}-\mathrm{J}} = -0.0090$ for the fitted Beta-Binomial prior and $\Delta_{\mathrm{R\,fit}-\mathrm{J}} = -0.0081$ for the fitted Rasch prior (Figure 2); its item difficulties alone, at budget zero, reach a within-task AUC of 0.836. On the unseen test benchmarks it equals the fitted prior ($\Delta_{\mathrm{R\,off}-\mathrm{J}} = -0.0115$ against $\Delta_{\mathrm{R\,fit}-\mathrm{J}} = -0.0119$): nothing it knows about particular items or benchmark rates is available. This is the seen-benchmark check the design requires, and it separates two things that are easy to conflate: item-level estimates transfer only to the items they were fitted on, while the prior's spreads carry over to new benchmarks.

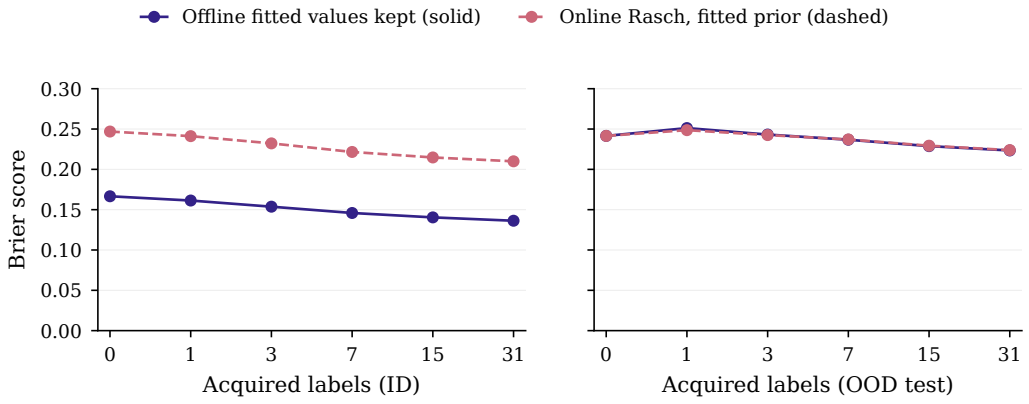


Figure 2: **Offline estimates win on familiar benchmarks and have no edge on unfamiliar ones.** Brier score per label budget for the offline Rasch keeping its fitted benchmark rates and item difficulties and updating only ability (solid), against the online Rasch with the same fitted prior (dashed). Left: ID, held-out subjects on the 11 training benchmarks. Right: OOD test benchmarks, where the offline model has no fitted values and falls back to the prior.

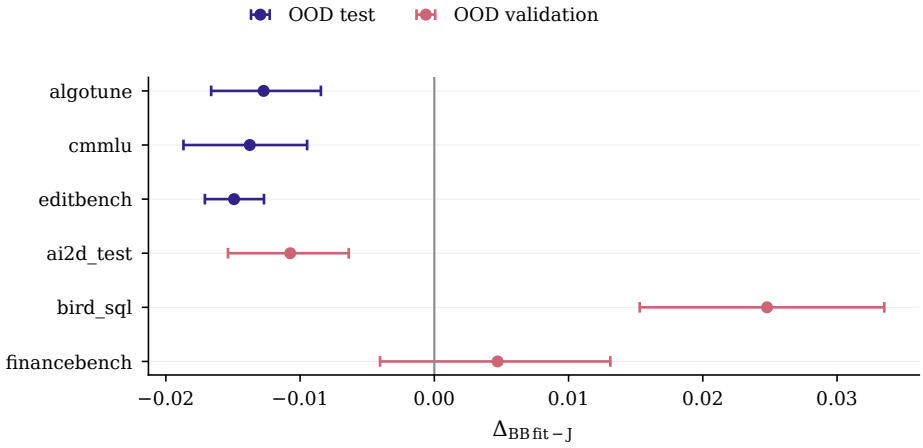


Figure 3: **The training-fitted prior helps on all three test benchmarks and on one validation benchmark, and hurts on the hardest.** $\Delta_{\mathrm{BB\,fit-J}}$ for each unseen benchmark, with 95% intervals over tasks. The validation benchmark `bird_sql` has task accuracy 0.12, far below every training benchmark.

5.3 Learned priors help benchmarks of typical difficulty and hurt much easier or harder ones

Figure 3 shows the test and validation benchmarks separately: all three test benchmarks agree ($\Delta_{\mathrm{BB\,fit-J}}$ from -0.013 to -0.015), and on validation `ai2d_test` agrees while `bird_sql` goes the other way ($+0.025$). Over the 17 benchmarks (Figure 4; each training benchmark is held out in turn, Appendix D) the fitted prior significantly helps on 9, significantly hurts on 1 (`bird_sql`), and the macro mean $\Delta_{\mathrm{BB\,fit-J}}$ is -0.0050. What separates the benchmarks is not whether they lie inside the range of training benchmark rates (`alignment_faking` and `harmbench` lose while inside it) but how far their task accuracies sit from the prior mean: the Spearman correlation between the gain and that distance is 0.96 over the 17 benchmarks (0.93 over the 11 training benchmarks), against 0.01 over the 11 for the benchmark's mean accuracy. The prior is a starting guess that a task is answered correctly about 52% of the time, and with few labels the predictor pulls its estimate toward it. If a benchmark's true accuracy is, say, 55%, the pull lands near the truth and the prediction improves; if

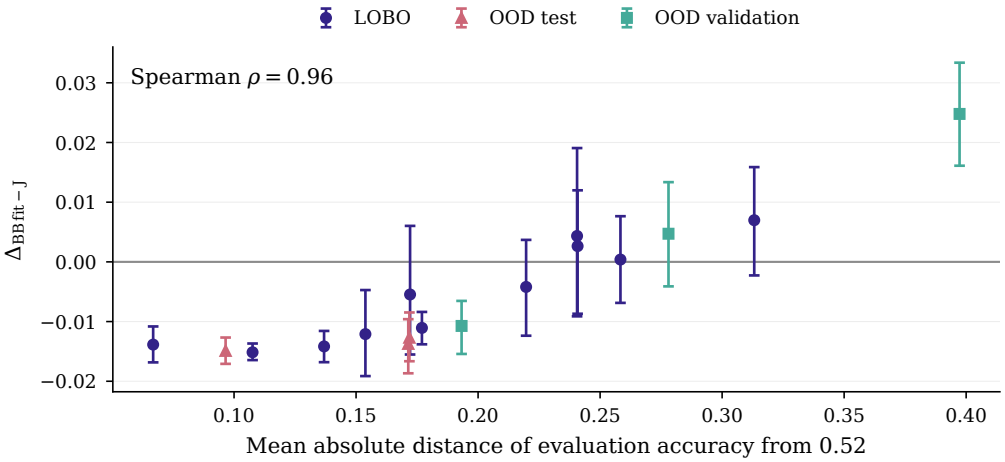


Figure 4: **A fitted prior helps when a benchmark's accuracy is near the prior mean.** Each point is one of 17 benchmarks. For each of the 11 training benchmarks, the prior was refit without any of that benchmark's rows and scored on held-out subjects. The six validation and test benchmarks use the original training-only prior. The horizontal axis uses task accuracy on the evaluation items; the vertical axis shows $\Delta_{\text{BB fit}-\text{J}}$, with 95% task-bootstrap intervals. This is an explanatory analysis using observed outcomes.

it is 12%, as on `bird_sql`, the pull drags the estimate the wrong way and it gets worse. So the prior helps on benchmarks of typical difficulty and hurts on much easier or much harder ones. The three test benchmarks (mean accuracy 0.54–0.68) sit near the prior mean, which is why test looks good. We found this pattern after seeing the results, so it is an explanation and not a predictor.

Under the fitted priors, the simplest Rasch model is also the best on test and the worst on validation: with benchmark rate only $\Delta_{m-\text{J}}$ is -0.0172 on test and $+0.0211$ on validation (Appendix A). The subject-ability term is what lets a benchmark far from the prior move away from it, at a cost in variance when the benchmark is near it. The kernel adds nothing on test.

5.4 Item content carries a small rank signal and a negligible gain for the prior

Table 3: Item-difficulty transfer: within-benchmark Spearman correlation and pairwise accuracy between predicted easiness and observed pass rate, averaged across benchmarks. ID: items held out within the 11 training benchmarks; OOD validation and OOD test: three unseen benchmarks each. The additional pass-rate and Rasch-target probes use benchmark-grouped selection and were not evaluated on ID (–).

	ID		OOD validation		OOD test	
Probe	Spearman	Pairwise	Spearman	Pairwise	Spearman	Pairwise
Random score	−0.003	0.499	−0.024	0.490	0.076	0.529
Log character count	0.107	0.539	0.058	0.525	0.061	0.521
Ridge on embeddings	0.400	0.649	0.196	0.570	−0.020	0.494
Pass-rate ridge	–	–	0.165	0.557	−0.035	0.487
Rasch-target ridge	–	–	0.130	0.544	−0.035	0.487
Pass-rate MLP	–	–	0.071	0.523	0.109	0.539
Rasch-target MLP	–	–	0.146	0.550	0.059	0.522

Embeddings show a clear signal on known benchmarks. On the six unseen benchmarks the two MLP probes pass the criterion we fixed in advance, with benchmark-macro Spearman 0.090 and 0.103 against 0.026 for a random baseline and a positive correlation on 5 of 6 benchmarks; the ridge probes do not (0.065 and 0.047). On the three test benchmarks alone, the pass-rate MLP reaches

0.109, while the Rasch-target MLP reaches 0.059, below the sampled random control's 0.076. The aggregate rank criterion therefore does not establish a consistent test-only advantage.

The rank signal yields only a small gain at the validation-selected scale. In Table 4, every $\gamma \geq 0.25$ is worse than $\gamma = 0$ on both splits. The scale selected on validation ($\gamma = 0.05$, Rasch-target MLP) improves Brier ALC by about 0.0004 on each split in its matched replay; this comparison alone does not establish a reliable improvement. This tests these two probes only; a stronger difficulty predictor, for instance one trained through an item response likelihood on more items, could do better.

Table 4: Predicted item difficulty as the prior mean of the Rasch item deviation, Equation (5). Cells are $\Delta_{m-\mathrm{J}}$ of the fitted Rasch with scale γ, competition-shaped draws; $\gamma = 0$ is the fitted Rasch without a prior mean. Bold: lowest in each column.

	Pass-rate MLP		Rasch-target MLP	
Scale γ	OOD validation	OOD test	OOD validation	OOD test
0	$\mathbf{+0.0052}$	-0.0119	$+0.0052$	-0.0119
0.05	$+0.0062$	$\mathbf{-0.0119}$	$\mathbf{+0.0048}$	$\mathbf{-0.0123}$
0.1	$+0.0103$	-0.0082	$+0.0051$	-0.0120
0.25	$+0.0332$	$+0.0150$	$+0.0099$	-0.0079
0.5	$+0.0690$	$+0.0491$	$+0.0273$	$+0.0069$
1	$+0.1065$	$+0.0826$	$+0.0704$	$+0.0433$
1.5	$+0.1326$	$+0.1050$	$+0.1040$	$+0.0724$

5.5 Priors that adapt to the first labels do not remove the harm

Table 5 compares the adaptive priors of Section 4.2 with the fitted Beta-Binomial prior over the 17 benchmarks. They trade gain for safety and none dominates it. The mixtures give the largest average and test gains but leave the worst benchmark as bad as before. Averaging over the mean and strength nearly removes the worst case but halves the gain. The switch rule falls in between. We had fixed three criteria in advance: a worst benchmark of at most $+0.0120$, a mean within 0.0010 of the fitted prior's -0.0050, and a test gain within 0.0020 of its -0.0138. No adaptive prior meets all three.

Table 5: $\Delta_{m-\mathrm{J}}$ for adaptive priors. Worst benchmark: the largest gain (a loss) over the 17 benchmarks. Mean: over the 17 benchmarks. The training benchmarks are held out one at a time, and the switch threshold is chosen on validation. Mixtures with 2 and 3 components differ by at most 0.0001 (the table shows 2). Bold: best of the adaptive priors in each column.

Model m	Worst benchmark	Mean over 17 benchmarks	OOD test
Fitted Beta-Binomial (reference)	$+0.0248$	-0.0050	-0.0138
Beta mixture, 2 or 3 components	$+0.0269$	$\mathbf{-0.0054}$	$\mathbf{-0.0163}$
Average over strength	$+0.0122$	-0.0045	-0.0110
Average over mean and strength	$\mathbf{+0.0034}$	-0.0037	-0.0063
Switch to Jeffreys, 3 labels	$+0.0136$	-0.0027	-0.0078
Switch to Jeffreys, 7 labels	$+0.0202$	-0.0042	-0.0114

Can the first labels warn that the prior will hurt? The distance of the accuracy so far from the prior's mean separates the tasks the prior hurts from the rest, with an area under the ROC curve (AUC) of 0.73 from seven labels, but only 0.64 within a benchmark, so most of the signal is between benchmarks. The harm sits at zero to three labels, which carry half of the ALC weight, before any such warning exists.

6 Limitations

The filtered public data covers only three test benchmarks in the online simulation. Multiple draws and acquisition orders measure variation within this population; they do not supply independent tests of transfer to new benchmarks. Task bootstrap intervals likewise do not capture uncertainty from sampling new benchmarks. Results apply to the eligible response data and random label acquisition, and need not carry over to other benchmark mixes or selection policies. The 17-benchmark

sensitivity analysis reuses held-out-subject tasks for the 11 original training benchmarks. The test triples overlap, so their distribution describes sensitivity within this dataset rather than independent estimates of transfer. The ID comparison has item response history that is unavailable under benchmark cold start. Observed item difficulty also depends on which subjects answered the item. The leave-one-benchmark-out association uses realized outcomes to explain when the fitted prior helps; it is not a way to select a prior before observing a new benchmark.

Further limits of our own. Each competition-shaped draw has one subject per benchmark, so sharing labels across subjects is not tested. These findings concern the models and data studied here, not a general impossibility of item-level transfer. The test gain depends on this split. The test benchmarks are close to the prior mean; over all 680 choices of three held-out benchmarks the fitted prior gains less ($\Delta_{\mathrm{BB\,fit-J}} = -0.0037$ on average, refit on the other 14) and hurts in 17.5% of them (Appendix E), as it does on validation. Test scores were inspected repeatedly while building the models, so the test analyses are exploratory: all selection used validation, but test was not read once. The kernel settings were chosen on validation and not on the ID evaluation set. We did not rerun the subject-metadata prior in this design, and we dropped reasoning-trace features, which are not available at test time.

7 Conclusion

What transfers from offline fitting to an unseen benchmark is a shrinkage prior, not item-level estimates. Where fitted values apply, offline fitting is worth a very large gain ($\Delta_{\mathrm{R\,off-J}} = -0.0852$ on the ID evaluation set); where they do not, it is worth none. The prior's gain is modest and depends on the benchmark: -0.0050 on average over the 17 benchmarks and -0.0037 over the 680 choices of three held-out benchmarks, with a loss in 17.5% of them. The prior helps when a benchmark's accuracy lies near the prior mean and hurts when it lies far away, which is why the same fitted prior looked good on test and bad on validation. Whether sharing labels across subjects helps remains untested, since each draw has one subject per benchmark. Two attempts to do better did not help: priors that adapt to the first labels did not remove the harm without losing the gain, and the neural probes we trained on item embeddings, which carry a small rank signal for difficulty, improved prediction by only about 0.0004 Brier ALC as a prior mean.

Disclosure

Training data. All fitted quantities (the offline Rasch fit, the Beta-Binomial empirical-Bayes prior, the item probes) use only the responses of the 11 training benchmarks in the competition's measurement database after the filtering described in Section 3; no other response data is used. **Pretrained models.** Item text is embedded with the frozen BGE-M3 model; nothing else is pretrained or fine-tuned. **LLM coding assistants.** Claude Code (Anthropic) assisted with writing code, analysis scripts and drafting text; every reported number was produced by running the code in the repository, and the authors checked the numbers. OpenAI Codex assisted with the reviewer analyses, figures, and text revisions.

References

[1] HELM african MMLU and WinoGrande, version 1.1.0. Stanford Center for Research on Foundation Models benchmark release. URL https://crfm.stanford.edu/helm/mmlu-winogrande-afr/latest/. Accessed 7 November 2026; distinct from the general HELM framework paper.

[2] David Ifeoluwa Adelani, Jade Abbott, Graham Neubig, Daniel D'souza, Julia Kreutzer, Constantine Lignos, Chester Palen-Michel, Happy Buzaaba, Shruti Rijhwani, Sebastian Ruder, Stephen Mayhew, Israel Abebe Azime, Shamsuddeen H. Muhammad, Chris Chinenye Emezue, Joyce Nakatumba-Nabende, Perez Ogayo, Aremu Anuoluwapo, Catherine Gitau, Derguene Mbaye, Jesujoba Alabi, Seid Muhie Yimam, Tajuddeen Rabiu Gwadabe, Ignatius Ezeani, Rubungo Andre Niyongabo, Jonathan Mukiibi, Verrah Otiende, Iroro Orife, Davis David, Samba Ngom, Tosin Adewumi, Paul Rayson, Mofetoluwa Adeyemi, Gerald Muriuki, Emmanuel Anebi, Chiamaka Chukwuneke, Nkiruka Odu, Eric Peter Wairagala, Samuel Oyerinde, Clemencia Siro, Tobius Saul Bateesa, Temilola Oloyede, Yvonne Wambui, Victor Akinode, Deborah Nabagereka, Maurice Katusiime, Ayodele Awokoya, Mouhamadane MBOUP, Dibora Gebreyohannes, Henok Tilaye, Kelechi Nwaike, Degaga Wolde, Abdoulaye Faye, Blessing Sibanda, Orevaoghene Ahia, Bonaventure F. P. Dossou, Kelechi Ogueji, Thierno Ibrahima DIOP, Abdoulaye Diallo, Adewale Akinfaderin, Tendai Marengereke, and Salomey Osei. MasakhaNER: Named entity recognition for African languages. *Transactions of the Association for Computational Linguistics*, 9:1116–1131, 2021. doi: 10.1162/tacl_a_00416. URL https://aclanthology.org/2021.tacl-1.66/.

[3] David Ifeoluwa Adelani, Graham Neubig, Sebastian Ruder, Shruti Rijhwani, Michael Beukman, Chester Palen-Michel, Constantine Lignos, Jesujoba O. Alabi, Shamsuddeen H. Muhammad, Peter Nabende, Cheikh M. Bamba Dione, Andiswa Bukula, Rooweither Mabuya, Bonaventure F. P. Dossou, Blessing Sibanda, Happy Buzaaba, Jonathan Mukiibi, Godson Kalipe, Derguene Mbaye, Amelia Taylor, Fatoumata Kabore, Chris Chinenye Emezue, Anuoluwapo Aremu, Perez Ogayo, Catherine Gitau, Edwin Munkoh-Buabeng, Victoire Memdjokam Koagne, Allahsera Auguste Tapo, Tebogo Macucwa, Vukosi Marivate, Elvis Mboning, Tajuddeen Gwadabe, Tosin Adewumi, Orevaoghene Ahia, Joyce Nakatumba-Nabende, Neo L. Mokono, Ignatius Ezeani, Chiamaka Chukwuneke, Mofetoluwa Adeyemi, Gilles Q. Hacheme, Idris Abdulmumin, Odunayo Ogundepo, Oreen Yousuf, Tatiana Moteu Ngoli, and Dietrich Klakow. MasakhaNER 2.0: Africa-centric transfer learning for named entity recognition. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang, editors, *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 4488–4508, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.298. URL https://aclanthology.org/2022.emnlp-main.298/.

[4] Tuka Alhanai, Adam Kasumovic, Mohammad Ghassemi, Aven Zitzelberger, Jessica Lundin, and Guillaume Chabot-Couture. Bridging the gap: Enhancing llm performance for low-resource african languages with new benchmarks, fine-tuning, and cultural adjustments, 2024. URL https://arxiv.org/abs/2412.12417.

[5] Anastasios N. Angelopoulos, Stephen Bates, Clara Fannjiang, Michael I. Jordan, and Tijana Zrnic. Prediction-powered inference. *Science*, 382(6671):669–674, 2023.

[6] Lucas Bandarkar, Davis Liang, Benjamin Muller, Mikel Artetxe, Satya Narayan Shukla, Donald Husa, Naman Goyal, Abhinandan Krishnan, Luke Zettlemoyer, and Madian Khabsa. The belebele benchmark: a parallel reading comprehension dataset in 122 language variants, 2024. URL https://arxiv.org/abs/2308.16884.

[7] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. BGE M3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. arXiv:2402.03216, 2024.

[8] Wayne Chi, Valerie Chen, Ryan Shar, Aditya Mittal, Jenny Liang, Wei-Lin Chiang, Anastasios Nikolas Angelopoulos, Ion Stoica, Graham Neubig, Ameet Talwalkar, and Chris Donahue. Edit-bench: Evaluating llm abilities to perform real-world instructed code edits, 2025. URL https://arxiv.org/abs/2511.04486.

[9] Bradley Efron and Carl Morris. Stein's estimation rule and its competitors—an empirical Bayes approach. *Journal of the American Statistical Association*, 68(341):117–130, 1973.

[10] Johannes Gasteiger, Vlad Mikulik, Hoagy Cunningham, Misha Wagner, Benjamin Wright, Jonathan Uesato, Joe Benton, Monte MacDiarmid, Fabien Roger, and Evan Hubinger. Towards training-time mitigations for alignment faking in RL. Anthropic Alignment Science Blog, December 2025. URL https://alignment.anthropic.com/2025/alignment-faking-mitigations/. Published 16 December 2025; source study for the alignment-faking-rl response release.

[11] Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. Cruxeval: A benchmark for code reasoning, understanding and execution, 2024. URL https://arxiv.org/abs/2401.03065.

[12] Tianrui Guan, Fuxiao Liu, Xiyang Wu, Ruiqi Xian, Zongxia Li, Xiaoyu Liu, Xijun Wang, Lichang Chen, Furong Huang, Yaser Yacoob, Dinesh Manocha, and Tianyi Zhou. Hallusionbench: An advanced diagnostic suite for entangled language hallucination and visual illusion in large vision-language models, 2024. URL https://arxiv.org/abs/2310.14566.

[13] Tianyu Guo, Hongyu Chen, Hao Liang, Meiyi Qiang, Bohan Zeng, Linzhuang Sun, Bin Cui, and Wentao Zhang. Brace: A benchmark for robust audio caption quality evaluation, 2025. URL https://arxiv.org/abs/2512.10403.

[14] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding, 2021. URL https://arxiv.org/abs/2009.03300.

[15] Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. Financebench: A new benchmark for financial question answering, 2023. URL https://arxiv.org/abs/2311.11944.

[16] Harold Jeffreys. An invariant form for the prior probability in estimation problems. *Proceedings of the Royal Society A*, 186, 1946.

[17] Minjeong Jeon, Ick Hoon Jin, Michael Schweinberger, and Samuel Baugh. Mapping unobserved item–respondent interactions: A latent space item response model with interaction map. *Psychometrika*, 86(2): 378–403, 2021.

[18] Aniruddha Kembhavi, Mike Salvato, Eric Kolve, Minjoon Seo, Hannaneh Hajishirzi, and Ali Farhadi. A diagram is worth a dozen images, 2016. URL https://arxiv.org/abs/1603.07396.

[19] Insung Lee, Taeyoung Jeong, Haejun Yoo, Du-Seong Chang, and Myoung-Wan Koo. Caf-score: Calibrating clap with lalms for reference-free audio captioning evaluation, 2026. URL https://arxiv.org/abs/2603.19615.

[20] Haonan Li, Yixuan Zhang, Fajri Koto, Yifei Yang, Hai Zhao, Yeyun Gong, Nan Duan, and Timothy Baldwin. Cmmlu: Measuring massive multitask language understanding in chinese, 2024. URL https://arxiv.org/abs/2306.09212.

[21] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls, 2023. URL https://arxiv.org/abs/2305.03111.

[22] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Cosgrove, Christopher D. Manning, Christopher Ré, Diana Acosta-Navas, Drew A. Hudson, Eric Zelikman, Esin Durmus, Faisal Ladhak, Frieda Rong, Hongyu Ren, Huaxiu Yao, Jue Wang, Keshav Santhanam, Laurel Orr, Lucia Zheng, Mert Yuksekgonul, Mirac Suzgun, Nathan Kim, Neel Guha, Niladri Chatterji, Omar Khattab, Peter Henderson, Qian Huang, Ryan Chi, Sang Michael Xie, Shibani Santurkar, Surya Ganguli, Tatsunori Hashimoto, Thomas Icard, Tianyi Zhang, Vishrav Chaudhary, William Wang, Xuechen Li, Yifan Mai, Yuhui Zhang, and Yuta Koreeda. Holistic evaluation of language models, 2023. URL https://arxiv.org/abs/2211.09110.

[23] G. Lior, T. Frostig, G. Stanovsky, and M. Eyal. Extending item response theory for efficient and meaningful multilingual evaluation. arXiv:2606.15643, 2026.

[24] Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 34303–34326. PMLR, 2024. URL https://proceedings.mlr.press/v235/maia-polo24a.html.

[25] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal, 2024. URL https://arxiv.org/abs/2402.04249.

[26] Charles Nimo, Tobi Olatunji, Abraham Toluwase Owodunni, Tassallah Abdullahi, Emmanuel Ayodele, Mardhiyah Sanni, Ezinwanne C. Aka, Folafunmi Omofoye, Foutse Yuehgoh, Timothy Faniran, Bonaventure F. P. Dossou, Moshood O. Yekini, Jonas Kemp, Katherine A Heller, Jude Chidubem Omeke, Chidi Asuzu Md, Naome A Etori, Aïmérou Ndiaye, Ifeoma Okoh, Evans Doe Ocansey, Wendy Kinara, Michael L. Best, Irfan Essa, Stephen Edward Moore, Chris Fourie, and Mercy Nyamewaa Asiedu. AfriMed-QA: A pan-African, multi-specialty, medical question-answering benchmark dataset. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1948–1973, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.96. URL https://aclanthology.org/2025.acl-long.96/.

[27] Ori Press, Brandon Amos, Haoyu Zhao, Yikai Wu, Samuel K. Ainsworth, Dominik Krupke, Patrick Kidger, Touqir Sajed, Bartolomeo Stellato, Jisun Park, Nathanael Bosch, Eli Meril, Albert Steppi, Arman Zharmagambetov, Fangzhao Zhang, David Perez-Pineiro, Alberto Mercurio, Ni Zhan, Talor Abramovich, Kilian Lieret, Hanlin Zhang, Shirley Huang, Matthias Bethge, and Ofir Press. Algotune: Can language models speed up general-purpose numerical programs?, 2025. URL https://arxiv.org/abs/2507.15887.

[28] J. N. K. Rao and Isabel Molina. *Small Area Estimation*. Wiley, 2nd edition, 2015.

[29] Georg Rasch. *Probabilistic Models for Some Intelligence and Attainment Tests*. Danish Institute for Educational Research, 1960.

[30] Herbert Robbins. An empirical Bayes approach to statistics. In *Proceedings of the Third Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 157–163, 1956.

[31] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale, 2019. URL https://arxiv.org/abs/1907.10641.

[32] Heinz Schmidli, Sandro Gsteiger, Satrajit Roychoudhury, Anthony O'Hagan, David Spiegelhalter, and Beat Neuenschwander. Robust meta-analytic-predictive priors in clinical trials with historical control information. *Biometrics*, 70(4):1023–1032, 2014. doi: 10.1111/biom.12242.

[33] Shuo Sun, Yimin Zhao, Christina Dao Wen Lee, Jiawei Sun, Chengran Yuan, Zefan Huang, Dongen Li, Justin KW Yeoh, Alok Prakash, Thomas W. Malone, and Marcelo H. Ang Jr. Agi-elo: How far are we from mastering a task?, 2025. URL https://arxiv.org/abs/2505.12844.

[34] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents, 2024. URL https://arxiv.org/abs/2407.18901.

[35] Sang T. Truong, Yuheng Tu, Percy Liang, Bo Li, and Sanmi Koyejo. Reliable and efficient amortized model-based evaluation. In *International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025.

[36] Guanhua Zhang, Florian E. Dorner, and Moritz Hardt. How benchmark prediction from fewer data misses the mark. arXiv:2506.07673, 2025.

A Rasch terms under the fitted priors

Table 6: $\Delta_{m-\mathrm{J}}$ for Rasch with terms switched off (prior width σ set near zero), under the training-fitted priors. Competition-shaped draws on OOD validation and OOD test. Bold: lowest in each column.

Model m	OOD validation	OOD test
Benchmark rate μ_d only	$+0.0211$ $[+0.0114, +0.0311]$	-0.0172 $[-0.0202, -0.0142]$
+ subject ability θ_i	$\mathbf{+0.0029}$ $[-0.0003, +0.0060]$	-0.0089 $[-0.0100, -0.0080]$
+ item deviation δ_j (Rasch)	$+0.0052$ $[+0.0007, +0.0098]$	-0.0119 $[-0.0133, -0.0106]$
+ kernel	$+0.0036$ $[-0.0006, +0.0078]$	-0.0118 $[-0.0131, -0.0105]$
μ_d and δ_j, no θ_i	$+0.0282$ $[+0.0164, +0.0401]$	$\mathbf{-0.0174}$ $[-0.0208, -0.0139]$
θ_i and δ_j, no μ_d	$+0.0078$ $[+0.0022, +0.0135]$	-0.0140 $[-0.0157, -0.0122]$

B Paired comparisons between prior sources

Table 7: Paired Δ_{a-b} between prior sources, with 95% intervals over tasks. Each comparison changes sign between OOD test and OOD validation. A dash: not run on ID.

Comparison	ID	OOD validation	OOD test
$\Delta_{\mathrm{R\,fit}-\mathrm{R\,fix}}$	–	$+0.0021$ $[+0.0006, +0.0036]$	-0.0017 $[-0.0028, -0.0005]$
$\Delta_{\mathrm{KR\,fit}-\mathrm{R\,fit}}$	–	-0.0016 $[-0.0021, -0.0012]$	$+0.0001$ $[-0.0002, +0.0004]$
$\Delta_{\mathrm{R\,fit}-\mathrm{BB\,fit}}$	$+0.0009$ $[-0.0001, +0.0018]$	-0.0027 $[-0.0041, -0.0012]$	$+0.0019$ $[+0.0007, +0.0031]$

C Draw-seed robustness

Table 8: $\Delta_{m-\mathrm{J}}$ on three fresh sets of 150 competition-shaped draws per role (seeds 20261201–03); results on the frozen draws are in Table 2.

Model m	OOD validation	OOD test
BB fit	$+0.0078$ / $+0.0083$ / $+0.0089$	-0.0137 / -0.0144 / -0.0139
R fit	$+0.0051$ / $+0.0054$ / $+0.0060$	-0.0120 / -0.0124 / -0.0123
BB tune	-0.0015 / -0.0016 / -0.0018	$+0.0060$ / $+0.0059$ / $+0.0063$

D The fitted prior on all 17 benchmarks

Table 9: $\Delta_{\mathrm{BB\,fit-J}}$ for each of the 17 benchmarks. Training benchmarks: the prior is refit without the held-out benchmark and scored on held-out-subject tasks. OOD validation and OOD test benchmarks: the original training-only prior. The last column is the mean distance of a benchmark's task accuracies from the prior mean 0.52.

Benchmark	Split	Tasks	$\Delta_{\mathrm{BB\,fit-J}}$ [95% CI]	Mean distance from 0.52
`hallusionbench` [12]	Training	20	-0.0151 [-0.0165, -0.0137]	0.11
`editbench` [8]	OOD test	17	-0.0149 [-0.0171, -0.0127]	0.10
`bridging_gap` [4, 14, 31, 6]	Training	20	-0.0142 [-0.0168, -0.0116]	0.14
`helm_afr` [4, 1, 22, 14, 31]	Training	3	-0.0139 [-0.0168, -0.0108]	0.07
`cmmlu` [20]	OOD test	10	-0.0137 [-0.0187, -0.0096]	0.17
`algotune` [27]	OOD test	18	-0.0127 [-0.0166, -0.0085]	0.17
`afrimedqa` [26]	Training	6	-0.0121 [-0.0191, -0.0047]	0.15
`afrieval` [2, 3]	Training	20	-0.0111 [-0.0138, -0.0084]	0.18
`ai2d_test` [18]	OOD validation	29	-0.0107 [-0.0154, -0.0065]	0.19
`cruxeval` [11]	Training	5	-0.0055 [-0.0155, $+0.0060$]	0.17
`brace` [13, 19]	Training	6	-0.0042 [-0.0124, $+0.0037$]	0.22
`agi_elo` [33, 14]	Training	4	$+0.0004$ [-0.0069, $+0.0076$]	0.26
`appworld` [34]	Training	4	$+0.0026$ [-0.0087, $+0.0120$]	0.24
`alignment_faking` [10]	Training	20	$+0.0043$ [-0.0091, $+0.0191$]	0.24
`financebench` [15]	OOD validation	16	$+0.0047$ [-0.0041, $+0.0134$]	0.28
`harmbench` [25]	Training	6	$+0.0070$ [-0.0023, $+0.0159$]	0.31
`bird_sql` [21]	OOD validation	35	$+0.0248$ [$+0.0161$, $+0.0334$]	0.40
Mean over the 17 benchmarks		239	-0.0050	0.20
Mean over the 11 training benchmarks		114	-0.0056	0.19
Spearman ρ between gain and distance: 0.96 over the 17 benchmarks, 0.93 over the 11 training benchmarks.				

E Sensitivity to the benchmark split

We held out each of the 680 triples, refitted the benchmark-weighted empirical Beta prior on the other 14 benchmarks, and rescored the frozen trajectories. The mean macro $\Delta_{\mathrm{BB\,fit-J}}$ was -0.0037, with 5th–95th percentiles [-0.0095, $+0.0099$]. The fitted prior hurt on 17.5% of splits. The original test benchmark names gave -0.0087 (percentile 13) under refitting. This refit includes the former validation benchmarks among the 14 eligible fit benchmarks for the original test triple, so its result differs from the original 11-benchmark fit. The 11 original training benchmarks are scored on held-out-subject tasks from the existing leave-one-out data. These overlapping triples describe split sensitivity within this dataset.

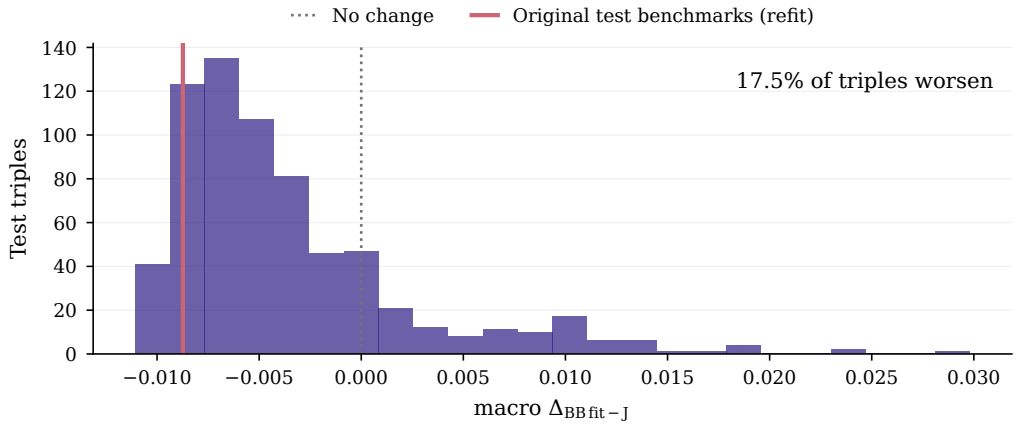


Figure 5: $\Delta_{\text{BB fit}-\text{J}}$ over all 680 triples. Each empirical prior is fitted on the 14 benchmarks outside its triple. The vertical line marks the original test benchmark names under this refitting protocol.

F Alternative evaluation metrics

Table 10: Competition-shaped online simulation; models are labelled as in Section 5. Brier, log loss, and ten-bin task ECE are averaged using the official budget weights; lower is better. AUC uses within-task item rankings at 31 labels, where higher is better. Dashes indicate no informative ranking; a constant prediction ties all items. Bold: best in each split (lowest loss, highest AUC). Per-budget AUC and alternative weights are reported in the accompanying CSVs.

Split	Model m	Brier	Log loss	ECE	AUC at 31 labels
ID	J	0.2357	0.6681	0.1407	–
	BB fit	0.2268	0.6437	0.1235	–
	R fit	0.2276	0.6460	0.1245	–
	KR fit	0.2295	0.6506	0.1336	0.581
	R off	**0.1506**	**0.4634**	**0.1161**	**0.836**
OOD validation	J	**0.1738**	**0.5159**	**0.1405**	–
	BB fit	0.1817	0.5404	0.1677	–
	R fit	0.1790	0.5323	0.1580	–
	KR fit	0.1774	0.5279	0.1603	**0.618**
OOD test	J	0.2499	0.7045	0.1426	–
	BB fit	**0.2362**	**0.6656**	**0.1113**	–
	R fit	0.2380	0.6712	0.1164	–
	KR fit	0.2382	0.6717	0.1230	**0.571**

G Solver convergence

Online solver. At every budget the online Rasch fit is a Newton solve with a limit of 30 steps. In 6,300 audited fits, none reached the limit, none needed more than 7 steps, and the last step was below 10^{-6} in every one.

Offline fit. Our first offline fit stopped at its iteration limit of 400 in three of its four rounds. We refit it with a proper stopping rule. The hyperparameters moved by at most 1.2%, the benchmark rates by at most 0.011 on the logit scale, and no Brier ALC changed by more than 0.0001. The reported results use the first fit, and the refit leaves them unchanged.

H Probe details

Training. Inputs are standardised using the fitting items only. The neural network has one hidden layer of 32 units and a limit of 100 epochs. The Rasch targets come from the offline Rasch fit.

From predictions to z_j. For the Rasch-target probe, the difficulty z_j is the negative of the predicted easiness. For the pass-rate probe, the centred prediction $f(x_j)$ is converted to a logit-scale difficulty,

$$z_j = \mathrm{logit}(\bar{p}) - \mathrm{logit}\big(\,\mathrm{clip}(\bar{p} + f(x_j),\, 10^{-4},\, 1 - 10^{-4})\big),$$

where $\bar{p}$ is the benchmark-weighted mean pass rate of the sampled training items. Positive difficulty means harder.